A Comparative Architectural Analysis of the Linux Kernel and Windows NT Architecture

Case Study

Submitted By:

Rohit Sharma Pandey

M.E Computer 1st Semester

Submitted in Partial Fulfillment of the Requirement of Master's Degree in ME Computer

Submitted to:

Everest Engineering College

Sanepa, Lalitpur-2



September 2026

Acknowledgement

I would like to express my sincere gratitude to **Everest Engineering College**, affiliated to **Pokhara University** for providing me with an opportunity to pursue my Master's degree in **ME COMPUTER**. I am grateful to the faculty members who have imparted their knowledge and skills throughout my academic journey.

I extend my heartfelt thanks to **Er. Narayan Sapkota**, the co-ordinator of the **ME COMPUTER department**, and respected **Asst. Professor Bishwadeep Mainaly** for his invaluable guidance and support during the preparation of my thesis proposal. His insights and suggestions have helped me shape my research ideas and direction.

I also appreciate the efforts of **the library staff and technical support team at Everest Engineering College** for providing me with access to the necessary resources and infrastructure to carry out my research.

Finally, I would like to express my gratitude to **my family and friends** for their constant encouragement and motivation. Their unwavering support has been instrumental in my academic and personal growth.

Thank you.

Your Sincerely
Rohit Sharma

Abstract

This paper compares the architectures of the Linux kernel and Windows NT, the two dominant general-purpose operating system designs in production use today. The comparison covers seven areas: kernel type, system call mechanism, process and thread management, file system architecture (ext4 versus NTFS), memory management, the device driver framework, and security models (Linux Security Modules versus the Windows Security Reference Monitor), along with inter-process communication (IPC). Linux uses a monolithic, modular kernel in which scheduling, memory management, networking, and file systems run in a single privileged address space extended by loadable kernel modules. Windows NT instead uses a hybrid architecture that separates a small microkernel and Hardware Abstraction Layer from a larger Executive that still runs in kernel mode. The two designs reach similar practical outcomes — preemptive multitasking, virtual memory, mandatory access control, and rich IPC — through different abstractions, and those differences show up in performance predictability, extensibility, and attack surface. The paper closes with a comparison of the performance and security trade-offs of each design and a consolidated comparison table.

Keywords

Linux kernel; Windows NT; monolithic kernel; hybrid kernel; system calls; process management; ext4; NTFS; memory management; device drivers; Linux Security Modules; Security Reference Monitor; inter-process communication.

Table of Contents

Acknowledgement i
Abstract ii
Keywords ii
1. Introduction 1
2. Kernel Type: Monolithic, Hybrid, and Modular Design 1
2.1 Linux: A Modular Monolithic Kernel 1
2.2 Windows NT: A Hybrid Kernel 2
3. System Call Mechanism 2
3.1 Linux 3
3.2 Windows NT 3
3.3 Comparative Observations 5
4. Process and Thread Management Models 5
4.1 Process/Thread Abstractions 5
4.2 Scheduling 6
4.3 Process Creation Cost 6
5. File System Architecture: ext4 versus NTFS 6
5.1 ext4 6
5.2 NTFS 7
5.3 Comparative Observations 7
6. Memory Management Strategies 7
6.1 Linux Memory Management 7
6.2 Windows NT Memory Management 8
7. Device Driver Framework 8
7.1 Linux Driver Model 8
7.2 Windows Driver Model 9
7.3 Comparative Observations 9
8. Security Models: Linux Security Modules versus the Windows Security Reference Monitor 9
8.1 Linux: DAC plus Linux Security Modules 9
8.2 Windows: The Security Reference Monitor 10
8.3 Comparative Observations 10
9. Inter-Process Communication (IPC) 11
9.1 Comparative Observations 12
10. Performance and Security Implications 12

10.1 Performance 12
10.2 Security 13
11. Consolidated Comparative Table 13
12. Conclusion 14
References 16

Lists of Figures

Figure 1: Hybrid kernel Architecture. 2
Figure 2: Linux Architecture 3
Figure 4: Linux vs Windows NT 5
Figure 5: Mandatory access control and Discretionary Access Control 11

1. Introduction

An operating system kernel sits between user software and hardware: it schedules the CPU, manages memory, exposes storage through a file system, arbitrates device access, enforces security policy, and provides ways for cooperating processes to communicate. Linux, first released in 1991, follows the Unix tradition of a monolithic kernel that has since been made modular through loadable kernel modules. Windows NT, designed from 1989 onward by a team that included several veterans of Digital Equipment Corporation's VMS project, took a hybrid approach meant to combine microkernel-style modularity with monolithic-style performance [1].

Both systems now run on the same commodity x86-64 and ARM64 hardware, expose POSIX-like programming surfaces at various points (for example, the Windows Subsystem for Linux), and are often deployed side by side in cloud and enterprise environments. Their internal architectures, though, remain fundamentally different. That difference shapes how each system handles workload variation, how device manufacturers write drivers, how security researchers reason about privilege boundaries, and how administrators tune performance. Comparative studies report measurable differences in multithreading and process-creation throughput between the two systems even on comparable hardware [3].

This paper draws on the operating systems literature — including foundational USENIX and ACM papers on the Linux Security Modules framework [4], [5], [6], the ext4 file system [7], [8], the Linux Completely Fair Scheduler [12], [13], and Microsoft's own description of the NT kernel architecture [1] — to build a structured, section-by-section comparison across the seven architectural areas covered here, followed by a discussion of what each design choice implies for performance and security.

2. Kernel Type: Monolithic, Hybrid, and Modular Design

2.1 Linux: A Modular Monolithic Kernel

Linux is classically described as a monolithic kernel: the scheduler, virtual memory manager, virtual file system (VFS), network stack, and device drivers all execute in a single kernel address space at the highest privilege level (Ring 0 on x86), communicating through direct function calls rather than message passing. This avoids the context-switch and message-marshalling overhead of microkernels, at the cost of a larger trusted computing base — a bug in any subsystem can, in principle, corrupt kernel memory used by every other subsystem.

Since the early 1.2 kernel series, Linux has addressed the extensibility weaknesses traditionally associated with monolithic design through Loadable Kernel Modules (LKMs), which let device drivers, file systems, and even entire subsystems be compiled separately and inserted into (or removed from) the running kernel without a reboot. This makes Linux's kernel type more precisely described as a modular monolithic kernel: architecturally monolithic, since modules share the kernel's address space and privilege level, but operationally modular, since functionality is not compiled statically into a single image.

2.2 Windows NT: A Hybrid Kernel

Windows NT is built around a layered Executive (ntoskrnl.exe) sitting atop a small kernel and a Hardware Abstraction Layer (HAL). The layering is meant to isolate hardware-specific code and low-level scheduling/interrupt primitives from the higher-level Executive services — object management, I/O management, process/memory management, and the security reference monitor [1]. Above the Executive, protected subsystems such as the Win32 subsystem run in user mode and translate Win32 API calls into native NT API calls. Because the Executive, the kernel proper, and kernel-mode device drivers all run in Ring 0, Windows NT is usually classified as a hybrid kernel: it keeps the layered, message-oriented flavor of a microkernel design internally, while, in Linus Torvalds's assessment, still running essentially all core services in kernel mode much like a monolithic system.

NT's hybrid layering matters architecturally more than it does for raw performance: it constrains how Microsoft and third parties structure new kernel-mode code (drivers interact with the I/O manager through well-defined IRP-based interfaces rather than arbitrary function calls), without imposing the cross-address-space IPC costs of a "pure" microkernel such as Mach or QNX [2].

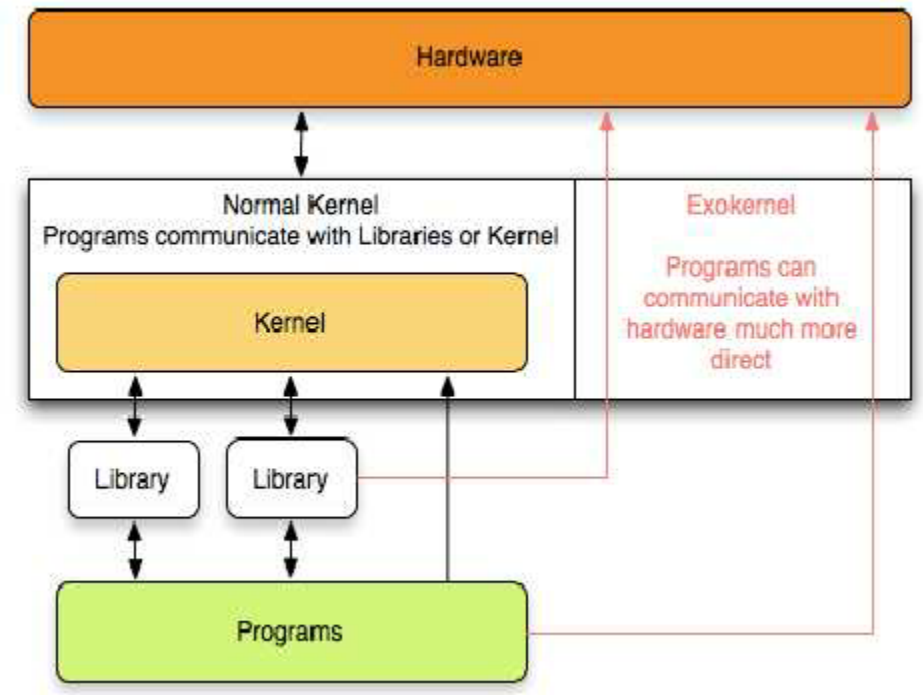


Figure 1: Hybrid kernel Architecture.

3. System Call Mechanism

Both kernels expose privileged services to user-mode code through a trap-based system call mechanism, but they differ in API surface, dispatch structure, and stability guarantees.

3.1 Linux

A Linux user process almost never issues a raw system call directly; instead it links against glibc (or musl, and so on), whose wrapper functions load arguments into registers and execute the SYSCALL instruction on x86-64 (historically INT 0x80 on 32-bit x86). This traps into entry_SYSCALL_64, which looks up the requested service in a single flat sys_call_table indexed by a syscall number, executes the corresponding kernel function (for example, vfs_read for read()), and returns via SYSRET [11]. Linux's syscall numbers and calling convention form a stable, versioned kernel-user ABI: the kernel developers' documented policy is that a syscall, once shipped, is never broken for existing binaries.

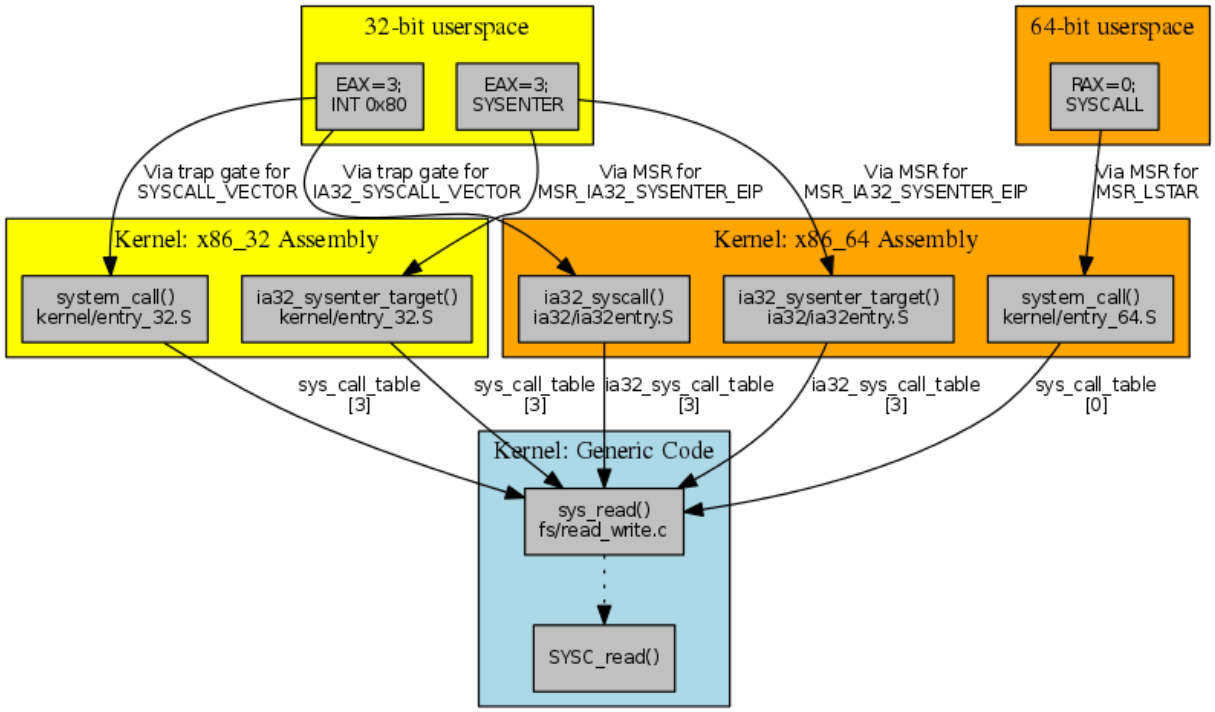


Figure 2: Linux Architecture

3.2 Windows NT

Windows applications typically call the documented Win32 API in kernel32.dll/advapi32.dll, which in turn calls the undocumented Windows Native API exposed by ntdll.dll (functions prefixed Nt, with Zw variants used internally by kernel-mode callers to skip redundant parameter validation). ntdll.dll issues the SYSCALL/SYSENTER instruction to trap into ntoskrnl.exe, which resolves the request through the System Service Dispatch Table (SSDT) — conceptually similar to Linux's sys_call_table, but reached through an additional user-mode translation layer and, unlike the Linux syscall ABI, not stable or documented for direct use across Windows versions [1].

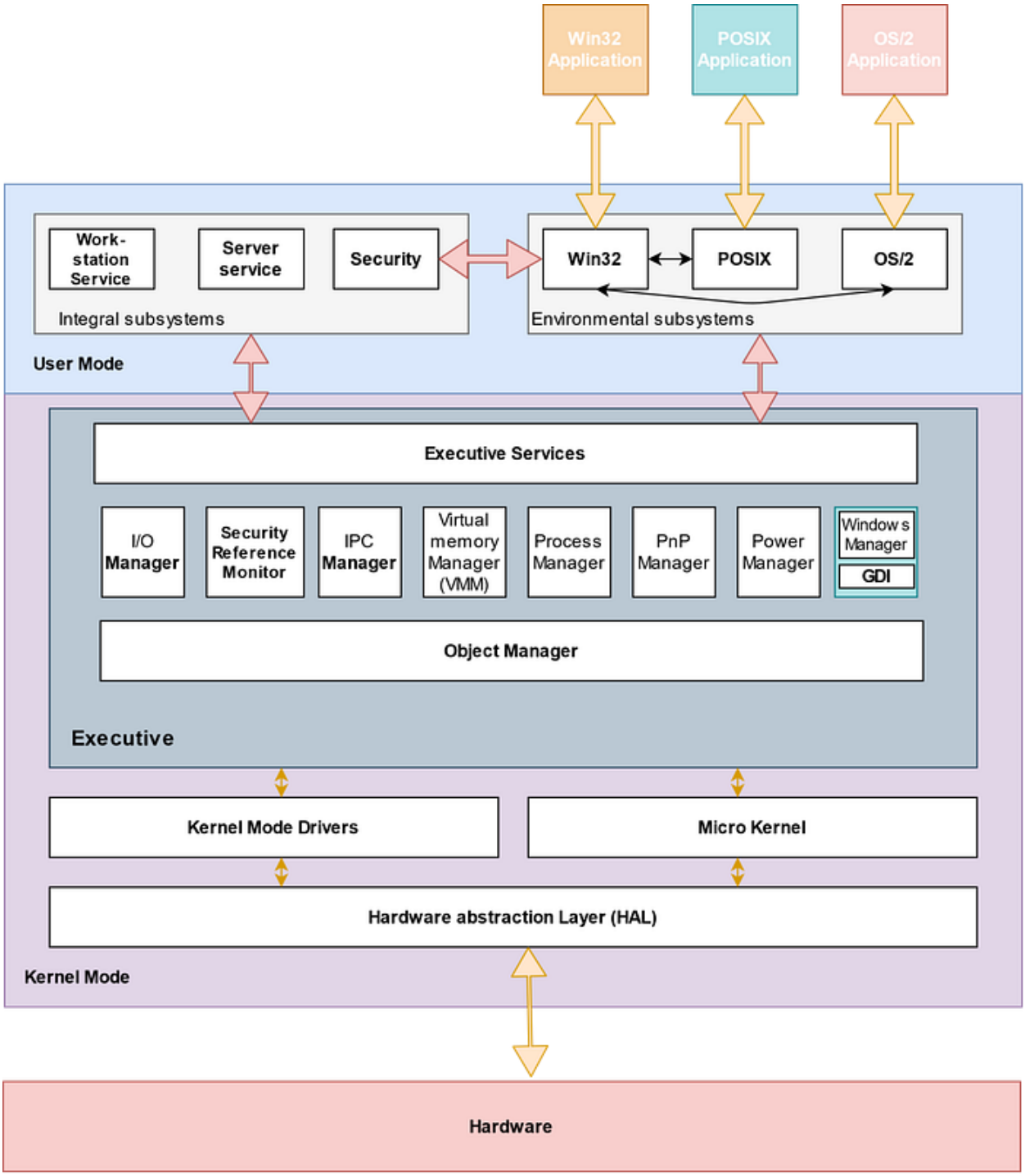


Figure 3: Window NT Architecture

3.3 Comparative Observations

- Linux exposes syscalls as a stable, public, numbered ABI; Windows exposes the Win32 API as the stable public surface while the underlying Native API/SSDT layout is version-dependent.
- Both use a single hardware trap instruction (SYSCALL/SYSENTER) and a flat dispatch table, so raw trap latency is comparable; NT's extra user-mode indirection through ntdll.dll adds one additional call frame versus glibc's more direct wrapper.
- Windows funnels virtually all resource access through kernel objects and handles managed by the Object Manager, so many "system calls" (for example, NtCreateFile) implicitly invoke the Security Reference Monitor, whereas Linux's DAC/LSM checks are embedded inline within the relevant VFS or IPC code path rather than a separate object layer.

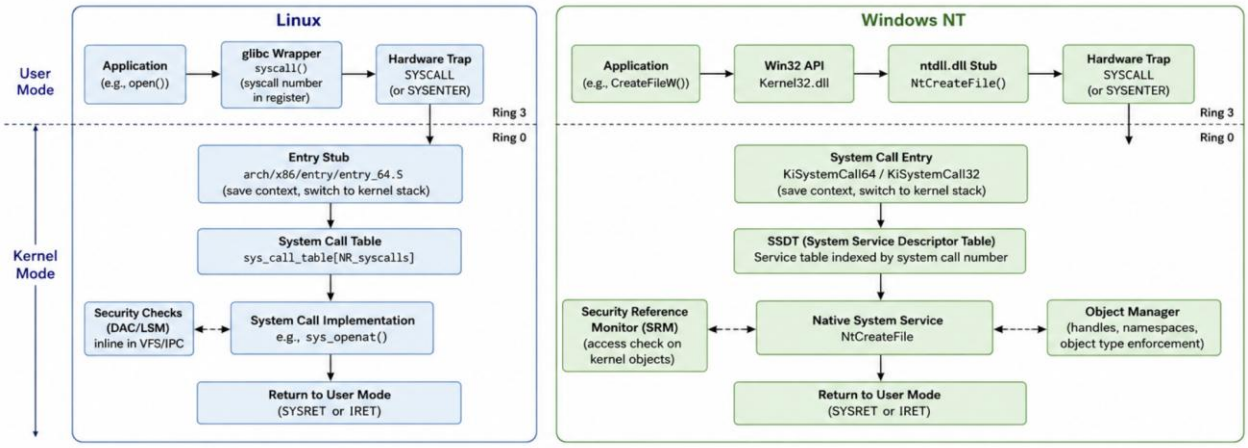


Figure 4: Linux vs Windows NT

4. Process and Thread Management Models

4.1 Process/Thread Abstractions

Linux represents every schedulable entity — process or thread — with a single data structure, task_struct, created through the clone() system call family; fork() and pthread_create() are both thin wrappers over clone() that differ only in which resources (address space, file descriptors, signal handlers) are shared between parent and child via clone flags such as CLONE_VM and CLONE_FILES. Because of this, the Linux scheduler treats processes and threads identically as "tasks," which simplifies the scheduler but requires userspace libraries (NPTL) to layer POSIX thread semantics on top.

Windows NT instead keeps explicit, separate kernel objects: an EPROCESS structure represents a process (an address space, a security context/access token, and a handle table), while one or more ETHREAD/KTHREAD structures represent the schedulable units within it. Threads, not processes, are scheduled; a process is purely a resource container. This separation makes certain operations — such as suspending or resuming an individual thread, or migrating thread state — more direct in the NT model, at the cost of two parallel object hierarchies that must be kept consistent.

4.2 Scheduling

Since kernel version 2.6.23, Linux's default scheduling class (SCHED_NORMAL/SCHED_OTHER) is the Completely Fair Scheduler (CFS), which orders runnable tasks in a red-black tree keyed by accumulated virtual runtime and always dispatches the leftmost (least-served) task, giving an approximation of ideal fair-share, O(log n) scheduling decisions [12]. Real-time classes (SCHED_FIFO, SCHED_RR) and, more recently, the EEVDF scheduler in newer kernels, run alongside CFS for latency-sensitive workloads. Empirical work has shown that naive fairness can still be gamed by spawning many threads, which has motivated refinements to CFS's group-scheduling weight distribution [12], and that core placement and idle-balancing heuristics can leave runnable threads waiting on idle cores under certain multicore topologies [13].

Windows NT uses a priority-driven, preemptive scheduler with 32 priority levels split into a real-time band (16-31) and a dynamic/variable band (0-15, plus a zero-page idle thread at 0). Threads in the dynamic band get temporary priority boosts after I/O completion or when a GUI thread is brought to the foreground, and their base priority decays back toward normal after receiving CPU time — a heuristic aimed at desktop interactive responsiveness rather than CFS's proportional-share fairness guarantee.

4.3 Process Creation Cost

Because clone()/fork() on Linux can share (rather than copy) an address space via copy-on-write and share file descriptor tables directly, thread and lightweight process creation is comparatively cheap. Comparative benchmark studies report substantially faster multithreading and process-creation times for Linux relative to Windows on equivalent hardware in Java-based microbenchmarks [3]. Windows process creation is heavier because CreateProcess() has to build a fresh EPROCESS, a virtual address space, an initial thread, and a security token, though thread creation within an existing process (CreateThread()) is considerably cheaper than process creation on either platform.

5. File System Architecture: ext4 versus NTFS

5.1 ext4

ext4 is the default file system on most Linux distributions and a direct, backward-compatible descendant of ext2/ext3. It was designed to remove ext3's 16 TiB volume ceiling and improve large-file performance while remaining mountable as ext3/ext2 where needed [7]. Its main architectural features include:

- Extent-based block mapping: instead of ext2/3's indirect block pointer trees, an extent describes up to 128 MB of physically contiguous blocks in a single metadata entry, which sharply reduces metadata overhead and seek cost for large files [7], [8].
- JBD2 metadata journaling: a write-ahead log (by default covering metadata only, data=ordered mode) that allows fast crash recovery without a full fsck [7].

- Delayed and multi-block allocation: block allocation decisions are deferred until writeback time, which improves on-disk contiguity and reduces fragmentation and allocation overhead compared to ext3's eager allocation [7].
- HTree-indexed directories, persistent preallocation, nanosecond timestamps, and 48-bit block addressing supporting volumes up to 1 EiB [7].

5.2 NTFS

NTFS (New Technology File System), introduced with Windows NT 3.1 in 1993, is a journaling, metadata-rich file system built around the Master File Table (MFT), a table of 1 KB records — one or more per file or directory — that store a file's attributes (name, timestamps, security descriptor, and either resident data or pointers to non-resident data runs) as typed attribute streams rather than a fixed inode layout. Every object in NTFS, including directories, is represented uniformly as a set of attributes, which is what allows features such as multiple named alternate data streams per file. Directory entries are organized in B-tree indexes for efficient lookup, and the file system supports Access Control Lists (ACLs) attached directly to each file's security descriptor, disk quotas, transparent compression, and encryption (EFS).

NTFS journals metadata operations (not file data by default) through its $LogFile using the Log File Service, which guarantees metadata consistency after an unclean shutdown. A companion $UsnJrnl (update sequence number journal) records a change history used by search indexing, backup, and replication tools rather than for crash recovery.

5.3 Comparative Observations

- Both file systems journal metadata by default rather than data, trading a small residual risk of data loss after a crash for lower steady-state I/O overhead; Linux additionally allows opting into full data journaling (data=journal), which NTFS does not expose as a mount-time choice.
- ext4's extent trees and NTFS's non-resident data runs solve the same problem — efficient mapping of large contiguous files — through structurally similar sparse-range representations.
- NTFS's uniform attribute model gives it native support for rich per-file metadata (ACLs, alternate streams, compression, encryption) directly in the file system; on Linux, equivalent functionality (POSIX ACLs, extended attributes, fscrypt) is layered on top of ext4 through the generic VFS extended-attribute interface rather than being intrinsic to ext4's on-disk format.
- Maximum theoretical volume/file sizes are comparable at modern scale — ext4 supports up to 1 EiB volumes and 16 TiB files; NTFS's original design supports volumes up to 256 TB in practice, with room in its 64-bit addressing for far larger sizes.

6. Memory Management Strategies

6.1 Linux Memory Management

Linux manages physical memory through a buddy allocator that services page-order (power-of-two) allocation requests and coalesces freed blocks to reduce fragmentation, layered underneath

the SLAB/SLUB/SLOB allocators that carve fixed-size kernel object caches (for example, for task_struct or inode objects) out of buddy-allocated pages. Each process's virtual address space is described by an mm_struct containing a list of Virtual Memory Areas (VMAs), each representing a contiguous, homogeneously-protected region (a memory-mapped file, the heap, a stack, an anonymous mmap region, and so on). Page reclaim is handled by the kswapd background daemon and, under memory pressure, by direct reclaim in the allocation path itself; if reclaim cannot free enough memory, the kernel's OOM killer selects and terminates a process using a heuristic "badness" score.

Linux supports memory overcommit (configurable via /proc/sys/vm/overcommit_memory), transparent huge pages, NUMA-aware allocation policies, and copy-on-write sharing of pages across fork()ed processes and memory-mapped files.

6.2 Windows NT Memory Management

The NT Memory Manager also implements demand-paged virtual memory but is organized around Virtual Address Descriptors (VADs) — a self-balancing tree per process describing reserved/committed virtual address ranges — and a working-set model that actively tracks and trims the set of physically resident pages per process through a balance-set manager, rather than relying mainly on a global reclaim daemon. Windows enforces stricter commit accounting than Linux's default overcommit posture: memory allocations are typically required to have backing (RAM or page file) reserved at commit time, and the system maintains a system-wide commit limit; when that limit is exhausted, allocation calls fail explicitly rather than the system as a whole invoking an OOM-killer-style process termination.

Both systems implement copy-on-write, memory-mapped files, and page-file/swap-backed anonymous memory, and both maintain page tables per the underlying architecture's paging hardware (multi-level page tables on x86-64/ARM64). The main difference is philosophical: Linux's reclaim path centers on a background/foreground scanning-and-eviction pipeline that ends, as a last resort, in an OOM killer, while NT's per-process working-set trimming and stricter commit accounting try to prevent global exhaustion from happening in the first place, at some cost to memory utilization efficiency under bursty, overcommitted workloads.

7. Device Driver Framework

7.1 Linux Driver Model

Linux device drivers are ordinarily compiled as Loadable Kernel Modules that link directly into the running kernel's address space and execute at full kernel privilege, registering with the appropriate subsystem (block, character, network, USB, PCI, platform) through that subsystem's driver-registration API. The unified device model, exposed to userspace via sysfs (/sys), organizes devices, drivers, and buses into a common object hierarchy with reference-counted kobjects, which enables hotplug (udev) and power-management integration across otherwise heterogeneous device classes. Because modules run in kernel space with no isolation from the rest of the kernel, a defective driver can corrupt kernel memory or crash the entire system; mainline Linux applies no mandatory code-signing requirement for module loading by default,

though distributions can enable module signature verification, particularly under UEFI Secure Boot.

7.2 Windows Driver Model

Windows drivers are written against the Windows Driver Model/Windows Driver Frameworks (WDM/WDF) and communicate with the I/O Manager through I/O Request Packets (IRPs) that flow down a layered driver stack (for example, a filter driver, then a function driver, then a bus driver for the same device). The Kernel-Mode Driver Framework (KMDF) still executes in Ring 0, while the User-Mode Driver Framework (UMDF) allows certain classes of drivers (many USB and imaging devices, for instance) to run in an isolated user-mode process, so a crash in the driver does not directly crash the kernel. Since Windows Vista/7, and enforced more strictly from Windows 10 onward, 64-bit Windows requires kernel-mode drivers to be digitally signed (kernel-mode code signing policy), and Driver Verifier and Kernel Patch Protection ("PatchGuard") further constrain what kernel-mode code — signed or not — is permitted to do to core kernel structures.

7.3 Comparative Observations

- Both frameworks layer drivers (Linux via subsystem-specific chains such as the SCSI/block layer stack; Windows via explicit IRP-passing driver stacks), but Windows formalizes the layering contractually through the IRP/IoCallDriver interface, while Linux relies on convention within each subsystem's function-pointer tables.
- Windows' UMDF and mandatory kernel-driver signing give it a stronger default posture against a single faulty or malicious driver compromising the whole kernel; Linux compensates through code review culture, optional signature enforcement, and increasing use of Rust-for-Linux and eBPF to move logic out of unsafe kernel C code, but does not mandate driver isolation by default.
- Linux's hotplug/sysfs model and Windows' Plug-and-Play (PnP) manager solve the same device-enumeration problem; PnP was a headline feature of the NT 5.0 (Windows 2000) Executive redesign [1].

8. Security Models: Linux Security Modules versus the Windows Security Reference Monitor

8.1 Linux: DAC plus Linux Security Modules

Linux's baseline access control is Discretionary Access Control (DAC): the classic owner/group/other permission bits (and POSIX ACLs as an extension), checked against the calling process's credentials. Because DAC alone cannot express confinement policies such as mandatory access control, the Linux Security Modules (LSM) framework was introduced to let the kernel support multiple security models "without bias" toward any one implementation [4]. LSM inserts hook calls at security-relevant points throughout the kernel — inode operations, file operations, process operations, socket/IPC operations, and more — each of which a loaded security module can inspect to allow or deny the operation, after standard DAC checks have already run [4], [6]. Concrete LSM-based modules include SELinux (type enforcement and multi-level security, originally developed by the NSA), AppArmor (path-based profiles), Smack,

TOMOYO, and the lightweight POSIX capabilities module; since Linux 4.2, several LSMs can be stacked concurrently rather than being mutually exclusive.

Getting these hooks placed correctly is itself a hard engineering problem: incomplete hook coverage can leave an operation unmediated. Static-analysis tooling such as TAHOE was built specifically to verify that every kernel operation requiring authorization is actually guarded by an LSM hook [5], and later measurement work has shown that the number of hooks nearly doubled between kernel 2.6.0 and 5.3 as LSM coverage expanded, while quantifying the associated performance overhead of policy enforcement on file operations [6].

8.2 Windows: The Security Reference Monitor

Windows NT centers its security architecture on the Security Reference Monitor (SRM), an Executive component that is the single, authoritative arbiter of every access to every securable object (files, registry keys, processes, threads, synchronization objects, and more), all represented uniformly by the Object Manager. Each subject (process or thread) carries an access token containing its user and group Security Identifiers (SIDs) and privileges; each object carries a security descriptor containing an owner SID, a Discretionary Access Control List (DACL) of Access Control Entries (ACEs) that grant or deny specific rights to specific SIDs, and an optional System Access Control List (SACL) that drives security auditing. Whenever a process requests a handle to an object, the SRM compares the token's SIDs against the object's DACL to compute the maximum allowed access, and, independently, checks the SACL to decide whether to emit an audit event [15].

This makes Windows access control fundamentally object-and-token-centric and centralizes policy evaluation in one Executive component, in contrast to Linux's hook-based approach, where policy evaluation is spread across many call sites in the kernel and delegated to pluggable modules.

8.3 Comparative Observations

Direct comparative analyses of the two models find that Windows' token/ACL scheme and Linux's DAC/LSM scheme both implement discretionary access control at their base layer. Linux additionally supports mandatory access control only when an LSM such as SELinux is enabled and correctly policied, whereas Windows' DACL model is present and enforced for every object by default, with Mandatory Integrity Control (integrity levels) layered on top for defense against untrusted processes such as browser sandboxes [15]. Windows centralizes both access decisions and audit generation in a single reference monitor, which makes it easier to reason about where access control happens; Linux distributes enforcement across many LSM hook sites, which maximizes flexibility and lets security policy evolve independently of the base kernel, at the cost of a larger and more intricate surface that must be kept correctly and completely hooked [4], [5], [6].

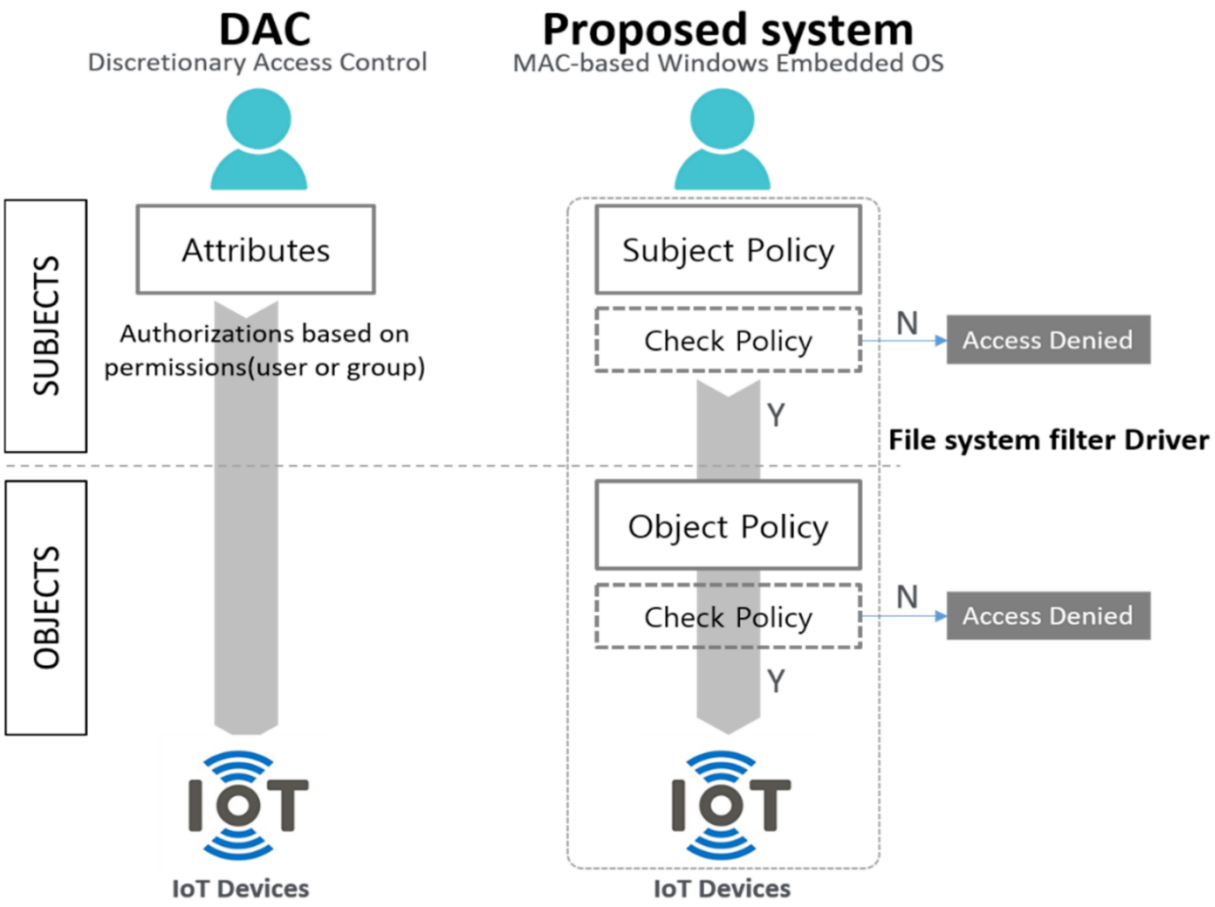


Figure 5: Mandatory access control and Discretionary Access Control

9. Inter-Process Communication (IPC)

Linux offers a wide, historically layered set of IPC primitives inherited from Unix and POSIX plus Linux-specific additions: anonymous pipes and named pipes (FIFOs) for byte-stream communication between related/unrelated processes; UNIX domain sockets for both byte-stream and datagram local communication (and, increasingly, as the transport for higher-level protocols such as D-Bus); the System V and POSIX shared-memory, semaphore, and message-queue families for higher-throughput or more structured communication; signals for lightweight asynchronous notification; and eventfd/futex primitives underpinning higher-level synchronization libraries. Shared memory combined with futex-based locking is the typical high-throughput path, since it avoids a kernel copy on every message; sockets and pipes are typically favored for their simpler, stream-oriented semantics and built-in flow control, at some cost to throughput.

Windows NT also offers multiple IPC layers, unified by the Executive's Advanced Local Procedure Call (ALPC) facility and the Object Manager's uniform handle model. Named pipes and mailslots offer stream- and datagram-oriented communication, respectively, usable both locally and over a network via the redirector; memory-mapped file objects (CreateFileMapping)

provide a POSIX-shared-memory equivalent with zero-copy sharing; and Windows sockets (Winsock) provide a BSD-socket-compatible networking API. Much of Windows' own subsystem infrastructure — Win32 subsystem calls, RPC transport, and many system services — is itself built on ALPC, a connection-oriented, message-passing facility integrated with the Object Manager and the SRM, so IPC endpoints are securable objects subject to the same DACL-based access control as files or registry keys. The higher-level Component Object Model (COM) and Distributed COM (DCOM)/RPC layer build on these primitives to provide structured, interface-based IPC both locally and across the network.

9.1 Comparative Observations

- Both systems provide a comparable menu of IPC mechanisms — pipes, sockets, shared memory, and message-based communication — reflecting similar engineering requirements rather than divergent ones.
- Windows' ALPC is more thoroughly integrated with its object/security model than any single Linux IPC primitive is with LSM, since every ALPC port is a securable Object Manager object evaluated by the SRM; Linux's IPC objects are protected more heterogeneously (System V IPC has its own permission bits; UNIX sockets rely on filesystem permissions of the socket path; LSMs must add dedicated hooks for IPC objects, for example, msg_queue_associate).
- Linux's signal mechanism has no precise Windows analogue; the closest functional equivalents on Windows (structured exception handling, Asynchronous Procedure Calls, and console control handlers) are more specialized and less uniformly available across arbitrary processes.

10. Performance and Security Implications

10.1 Performance

The monolithic-plus-modules design gives Linux short, direct call paths between subsystems (a syscall handler can call directly into the VFS, which calls directly into the block layer, with no message-passing indirection). Comparative kernel benchmarking points to this as a factor in Linux's generally faster raw process-creation and multithreading throughput relative to Windows on equivalent hardware [3]. The Completely Fair Scheduler's O(log n) red-black-tree scheduling and lightweight clone()-based thread/process creation reinforce this advantage for workloads dominated by many short-lived tasks, though later measurement work notes that CFS's per-core load-balancing heuristics can still leave runnable threads idle under specific multicore contention patterns, which continues to motivate scheduler research [13].

Windows NT's layered Executive and IRP-based driver stacks add extra indirection at I/O time, and its stricter working-set/commit-based memory model trades some raw allocation throughput for more predictable behavior under memory pressure (favoring graceful allocation failures over Linux's global OOM-killer intervention). In practice, both platforms have converged substantially at the hardware-interaction level (NUMA-aware scheduling, I/O completion ports versus io_uring/epoll, huge/large page support), so most of the residual performance gap reported in the literature is workload- and configuration-dependent rather than a fixed architectural verdict [3].

10.2 Security

Linux's DAC-plus-LSM model offers a lot of policy flexibility — an organization can choose SELinux's rigorous type enforcement, AppArmor's simpler path-based confinement, or no MAC at all — but that flexibility means the security posture of a given Linux system depends heavily on which LSM (if any) is enabled and how completely its hooks cover the kernel's attack surface. That completeness problem is serious enough to have motivated dedicated static-analysis tooling [5] and ongoing overhead/coverage measurement [6]. Windows' SRM, by contrast, enforces DACL-based access control uniformly and by default for every securable object, giving a more consistent security baseline out of the box, augmented by Mandatory Integrity Control and, on modern systems, kernel-mode code-signing and Kernel Patch Protection to reduce the risk that a single compromised or malicious driver can subvert the reference monitor itself.

Neither model is strictly better: Linux's approach shifts responsibility (and expressive power) to policy authors and distribution maintainers, while Windows' centralized reference-monitor approach concentrates both power and risk in one Executive component whose own compromise (for example, via a kernel-mode driver vulnerability) would defeat the entire access-control scheme at once. Comparative security analyses generally conclude that both architectures offer broadly comparable theoretical guarantees when correctly configured, with practical security differences driven more by patching cadence, default configuration, and driver/module trustworthiness than by the abstract access-control model itself [15].

11. Consolidated Comparative Table

Table 1 summarizes the architectural comparison developed in Sections 2 through 9.

Dimension	Linux	Windows NT
Kernel type	Monolithic, modular (loadable kernel modules extend a single privileged kernel image)	Hybrid (microkernel-influenced Executive + kernel-mode drivers, all in Ring 0/kernel mode) [1], [3]
System call entry	Flat sys_call_table indexed by a stable syscall number; entered via SYSCALL/SYSENTER trap [11]	Native Nt*/Zw* API resolved through the System Service Dispatch Table (SSDT) inside ntoskrnl.exe; Win32 API in user mode wraps it
Process abstraction	task_struct describes both processes and threads ("tasks"); lightweight clone() controls sharing	Separate EPROCESS/ETHREAD objects; processes are containers, threads are the schedulable unit [1]
CPU scheduling	Completely Fair Scheduler (CFS): red-black tree ordered by virtual runtime, O(log n) picks [12], [13]	Priority-driven, preemptive, 32-level dynamic/real-time priority scheduler with quantum boosting
Thread model	1:1 kernel threads; POSIX threads (NPTL) map directly to kernel scheduling entities	1:1 kernel threads; fibers available for user-mode cooperative scheduling atop a thread
File system	ext4: extent-based mapping, JBD2	NTFS: B-tree (B+) directory

Dimension	Linux	Windows NT
(default)	metadata journaling, delayed allocation, htree directories [7], [8]	indexing, Master File Table (MFT), $LogFile write-ahead journaling of metadata [9], [14]
Journaling scope	Metadata-only by default (data=ordered); full data journaling optional	Metadata-only journaling via the Log File Service (LFS); no full data journaling
Memory management	Buddy allocator for physical pages, SLAB/SLUB for kernel objects, per-process mm_struct with VMAs	Working-set based VM manager, Virtual Address Descriptors (VADs), Memory Manager with page-file backing
Overcommit / reclaim	Optional overcommit heuristics; kswapd and direct reclaim; OOM killer as last resort	No general overcommit by default; balance-set manager trims working sets; no OOM killer equivalent
Device drivers	Loadable Kernel Modules (LKMs) linked directly into kernel address space; unified driver model /sys	WDM/WDF framework; drivers layered (bus/function/filter) and can run in kernel or user mode (UMDF)
Driver signing / isolation	No mandatory signing on mainline Linux (distro-dependent, e.g. Secure Boot MOK); no driver sandboxing	Mandatory kernel-mode driver signing since Windows 10 (kernel-mode code signing policy); UMDF isolates some drivers in user mode
Security model	Discretionary Access Control (DAC) + stackable Linux Security Modules (SELinux, AppArmor, Smack) [4], [5], [6]	Security Reference Monitor (SRM): token-based subject, DACL/SACL-based object security descriptors [15]
Access-control granularity	Per-hook mandatory access control layered above DAC; policy language depends on module (e.g., SELinux TE)	Per-object ACL entries (ACEs) evaluated against the caller's access token; integrated auditing via SACL
Primary IPC mechanisms	Pipes/FIFOs, UNIX domain sockets, System V & POSIX shared memory/semaphores/message queues, signals, D-Bus	Named pipes, mailslots, Local/Remote Procedure Call (LPC/ALPC), Windows sockets, memory-mapped files, COM
Synchronous IPC	Signals (asynchronous, limited data) and blocking reads on pipes/sockets	ALPC provides synchronous, connection-oriented message passing tightly integrated with the Object Manager

Table 1. Consolidated architectural comparison of the Linux kernel and Windows NT.

12. Conclusion

Linux and Windows NT reach broadly similar functional capabilities — preemptive multitasking, virtual memory, journaling file systems, mandatory access control, and rich IPC —

by different architectural routes. Linux's modular monolithic kernel favors direct, low-latency subsystem interaction and a flexible, community-driven security model built on stackable LSMs, at the cost of a larger implicit trust boundary among in-kernel components. Windows NT's hybrid Executive favors structured layering (IRP-based drivers, an ALPC-mediated IPC fabric, and a single centralized Security Reference Monitor), which gives more uniform default security guarantees and clearer subsystem contracts, at some cost in indirection. In practice, the choice between them is rarely made on architectural grounds alone — ecosystem, driver availability, licensing, and operational familiarity usually decide it. Even so, the architectural trade-offs covered here — kernel type, system call dispatch, process/thread models, file system design, memory management philosophy, driver frameworks, security models, and IPC — matter for anyone who has to reason precisely about how either platform behaves under real workloads.

References

[1] Solomon, D. A. (1998). The Windows NT kernel architecture. *Computer*, *31*(10), 40–47. https://doi.org/10.1109/2.722284

[2] Kavas, D., Lancaster, D., Takeda, K., et al. (2002). Comparing Windows NT, Linux, and QNX as the basis for cluster systems. *Concurrency and Computation: Practice and Experience*, *14*(8-9), 671–687.

[3] Zeebaree, S. R. M., et al. (2016). A comparative evaluation of core kernel features of the recent Linux, FreeBSD, Solaris and Windows operating systems. *Asian Journal of Research in Computer Science*, *1*(1), 1–15.

[4] Smalley, S., Vance, C., & Salamon, W. (2002). Linux Security Modules: General security support for the Linux kernel. In *Proceedings of the 11th USENIX Security Symposium*. USENIX Association.

[5] Ganapathy, V., Jaeger, T., & Jha, S. (2005). Automatic placement of authorization hooks in the Linux Security Modules framework. In *Proceedings of the 12th ACM Conference on Computer and Communications Security (CCS '05)* (pp. 330–339). Association for Computing Machinery. https://doi.org/10.1145/1102120.1102164

[6] Zhang, W., Liu, P., & Jaeger, T. (2021). Analyzing the overhead of file protection by Linux Security Modules. In *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security (ASIA CCS '21)* (pp. 393–404). Association for Computing Machinery. https://doi.org/10.1145/3433210.3453078

[7] Mathur, A., Cao, M., Bhattacharya, S., Dilger, A., Tomas, A., & Vivier, L. (2007). The new ext4 filesystem: Current status and future plans. In *Proceedings of the Linux Symposium* (Vol. 2, pp. 21–33). Linux Symposium.

[8] Cao, M., Bhattacharya, S., & Tso, T. Y. (2007, March). *Ext4: The next generation of Ext2/3 filesystem* [Paper presentation]. 2007 Linux Storage & Filesystem Workshop (LSF '07), San Jose, CA, United States.

[9] Lee, C., Sim, D., Hwang, J., & Cho, S. (2015). F2FS: A new file system for flash storage. In *Proceedings of the 13th USENIX Conference on File and Storage Technologies (FAST '15)* (pp. 273–286). USENIX Association.

[10] U.S. Patent No. 6,314,437. (2001). *Method and apparatus for real-time secure file deletion*. U.S. Patent and Trademark Office.

[11] Love, R. (2010). *Linux kernel development* (3rd ed.). Addison-Wesley.

[12] Wong, C. S., Tan, I., Kumari, R. D., & Wey, F. (2008). Towards achieving fairness in the Linux scheduler. *ACM SIGOPS Operating Systems Review*, *42*(5), 34–43. https://doi.org/10.1145/1400097.1400102

[13] Lozi, J., Lepers, B., Funston, J., Gaud, F., Quema, V., & Fedorova, A. (2016). The Linux scheduler: A decade of wasted cores. In *Proceedings of the Eleventh European Conference on Computer Systems (EuroSys '16)*. ACM. https://doi.org/10.1145/2901318.2901326

[14] NTFS: Introduction and analysis from forensics point of view. (2023). In *Proceedings of the 2023 International Conference for Advancement in Technology (ICONAT)*. IEEE. https://doi.org/10.1109/ICONAT57137.2023.10080271

[15] Song, X., Stinson, M., Lee, R., & Albee, P. (2006). An approach to analyzing the Windows and Linux security models. In *Proceedings of the 5th IEEE/ACIS International Conference on Computer and Information Science (ICIS-COMSAR '06)*. IEEE. https://doi.org/10.1109/ICIS-COMSAR.2006.18